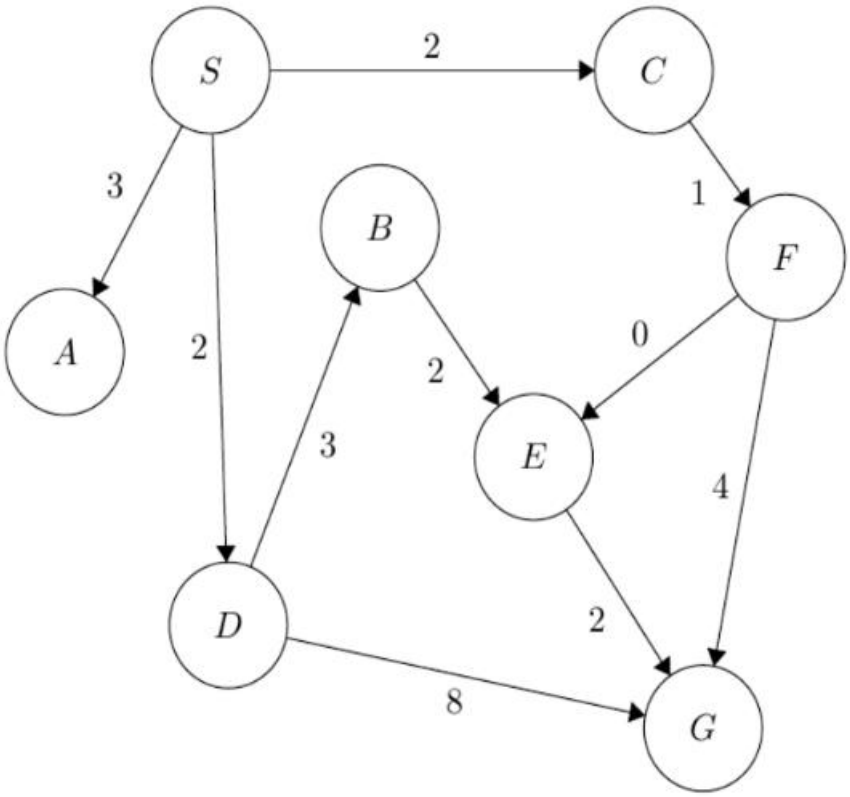
BÀI TẬP TÌM KIẾM TRONG AI

Họ và tên	Lê Đoàn Kim Ngân
MSSV / Lớp	3123410231 / DCT1236

Bài 1. Các thuật toán tìm kiếm



1.1. Breadth-First Search (BFS)

Nội dung	**Kết quả**
Explored Set	S, A, C, D, F, B, G
Path	S → D → G
Total Cost	2 + 8 = 10

1.2. Depth-First Search (DFS)

Nội dung	**Kết quả**
Explored Set	S, A, C, F, E, G
Path	S → C → F → E → G
Total Cost	2 + 1 + 0 + 2 = 5

1.3. Iterative Deepening Search (IDS)

Giới hạn độ sâu	**Các nút được duyệt**

Giới hạn độ sâu	Các nút được duyệt
0	S
1	S, A, C, D
2	S, A, C, F, D, B, G (G ở độ sâu 2 nên tìm thấy)

Path cuối cùng: S → D → G

Total Cost: 10

Câu hỏi thảo luận: BFS và DFS khác nhau ở cấu trúc frontier và tính tối ưu như thế nào?

BFS dùng queue (FIFO), mở rộng theo từng tầng, nên cho đường đi ít cạnh nhất. Nó chỉ tối ưu về chi phí khi mọi cạnh có cost bằng nhau (ở đây BFS ra cost 10 trong khi DFS ra 5). DFS dùng stack (LIFO), đi sâu trước, tốn ít bộ nhớ hơn nhưng không đảm bảo tối ưu. Trên đồ thị có chu trình hoặc không gian vô hạn, DFS còn có thể không hoàn chỉnh.

Bài 2. Tháp Hà Nội - Tower of Hanoi

2.1. Biểu diễn trạng thái

Dùng bộ n phần tử $(p_1, p_2, ..., p_n)$, trong đó $p_i \in \{0, 1, 2\}$ là cọc mà đĩa i đang nằm. Đĩa 1 nhỏ nhất, đĩa n lớn nhất. Trên mỗi cọc, các đĩa luôn xếp lớn dưới nhỏ trên, nên thứ tự trên cọc được suy ra từ kích thước, không cần lưu thêm.

2.2. Kích thước không gian trạng thái

Mỗi đĩa chọn 1 trong 3 cọc, độc lập với nhau, nên có $\mathbf{3^n}$ trạng thái. Mọi cách gán đều hợp lệ vì thứ tự xếp trên cọc đã cố định theo kích thước.

2.3. Trạng thái bắt đầu

$S_0 = (0, 0, ..., 0)$, tất cả n đĩa ở cọc 0.

2.4. Các hành động hợp lệ

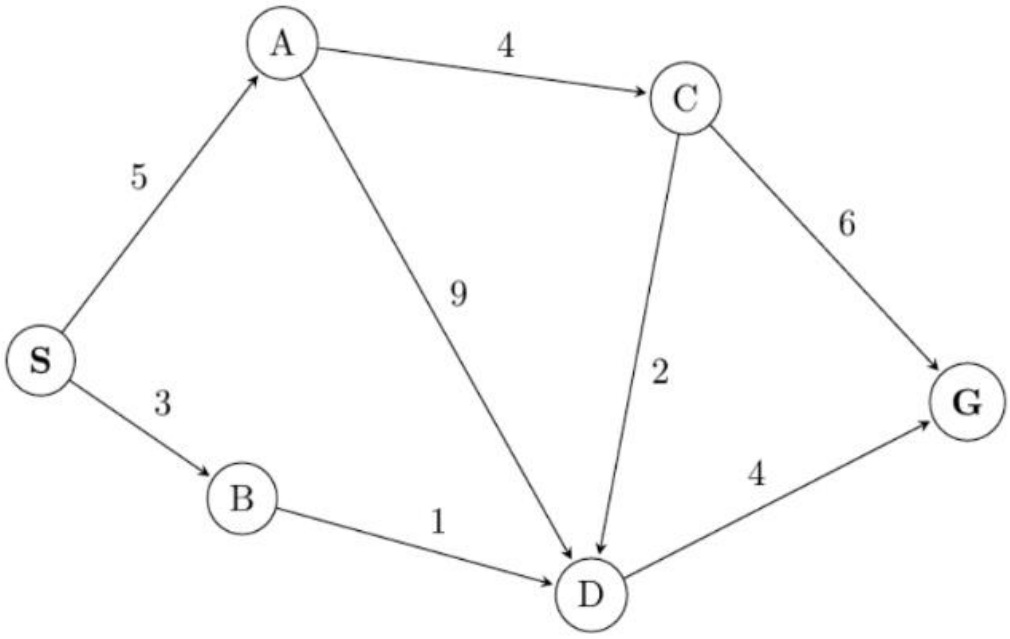
Mỗi hành động là (i, x → y), tức chuyển đĩa i từ cọc x sang cọc y. Điều kiện hợp lệ:

- $x \neq y$ và $p_i = x$ (đĩa i đang ở cọc x).
- Đĩa i là đĩa trên cùng của cọc x, tức không có đĩa nhỏ hơn i nằm ở cọc x.
- Cọc y không có đĩa nhỏ hơn i, để không đặt đĩa lớn lên đĩa nhỏ (cọc y rỗng cũng được).

2.5. Goal Test

goalTest(state) trả về True khi mọi đĩa ở cọc thứ ba, tức state = (2, 2, ..., 2), nghĩa là $p_i = 2$ với mọi i.

Bài 3. Thiết kế và phân tích Heuristic



3.1. Thiết kế heuristic admissible và consistent

Node	h(n)
S	8
A	10
B	5
C	6
D	4
G	0

- Admissible: mọi $h(n) \leq h^*(n)$.
- Consistent, kiểm tra từng cạnh:
  - S→A: $8 \leq 5+10$
  - S→B: $8 \leq 3+5$
  - A→C: $10 \leq 4+6$
  - A→D: $10 \leq 9+4$
  - B→D: $5 \leq 1+4$
  - C→G: $6 \leq 6$
  - C→D: $6 \leq 2+4$
  - D→G: $4 \leq 4$

3.2. Chạy A* với heuristic đã thiết kế

- Lấy S (f=8), sinh A (g=5, f=5+10=15) và B (g=3, f=3+5=8).
- Lấy B (8), sinh D (g=4, f=4+4=8).
- Lấy D (8), sinh G (g=8, f=8+0=8).
- Lấy G (8) thì dừng.

Explored Set: S, B, D, G

Path: S → B → D → G

Total Cost: 8

3.3. Heuristic admissible nhưng không consistent

Giữ nguyên h* cho các node khác, chỉ hạ D xuống: S=8, A=10, B=5, C=6, **D=1**, G=0

- Admissible vì mọi giá trị ≤ h* (D=1 ≤ 4).
- Cạnh vi phạm: B→D có h(B)=5 > c(B,D) + h(D) = 1+1 = 2. Cạnh C→D cũng vi phạm vì 6 > 2+1 = 3.

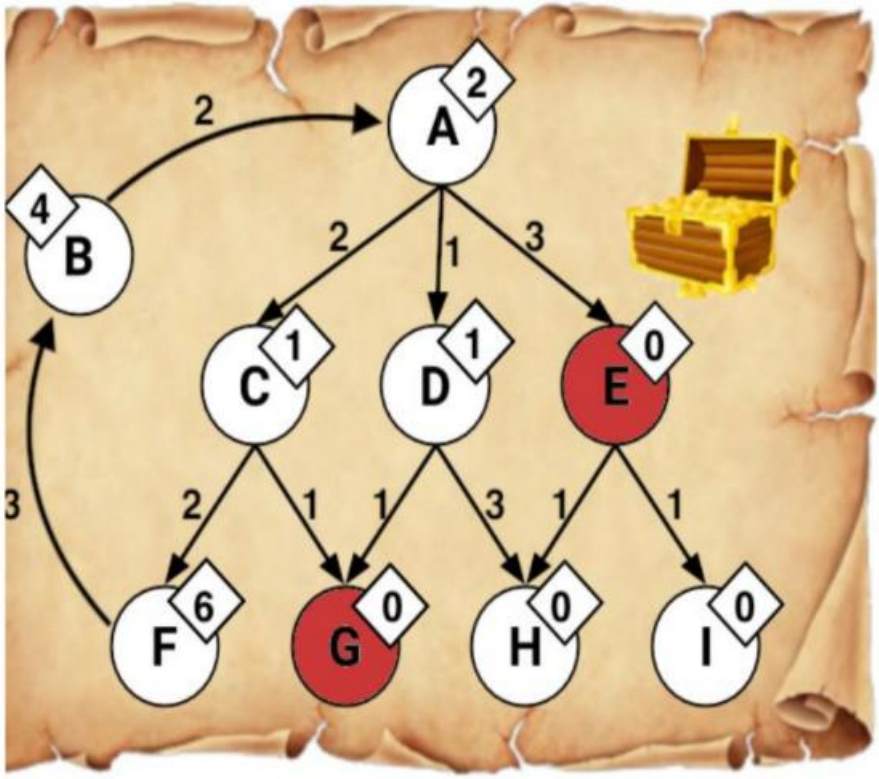
3.4. Câu hỏi mở rộng

Gọi đường tối ưu từ n đến đích là $n = n_0 \rightarrow n_1 \rightarrow ... \rightarrow n_k = G$. Áp dụng consistency dọc đường đi:

$h(n_0) \leq c(n_0,n_1) + h(n_1) \leq c(n_0,n_1) + c(n_1,n_2) + h(n_2) \leq ... \leq$ tổng chi phí đường đi + h(G)

Vì h(G) = 0 nên vế phải bằng h*(n). Vậy h(n) ≤ h*(n), tức là admissible.

Bài 4. Treasure Hunting (Tìm kho báu)



4.1. Depth-First Search
Explored Set: A, C, F, B, G

Path returned: A → C → G

4.2. Breadth-First Search
Explored Set: A, C, D, E

Path returned: A → E

4.3. Uniform-Cost Search

- Lấy D (g=1) trước, sinh G (g=2) và H (g=4).
- Sau đó C (g=2) và G (g=2) hòa nhau, chọn C theo alphabet. C sinh G với g=3 nên bỏ qua vì đã có g=2 tốt hơn.
- Lấy G (g=2) là kho báu thì dừng.

Explored Set: A, D, C, G

Path returned: A → D → G

Cost: 2

4.4. Greedy Search
Chọn theo h. Sau A, E có h=0 nhỏ nhất nên được lấy ngay và là kho báu.

Explored Set: A, E

Path returned: A → E

Cost: 3

4.5. A* Search

- A có f=2. Sinh C (f=3), D (f=2), E (f=3).
- Lấy D (2), sinh G (g=2, f=2) và H (f=4).
- Lấy G (2) thì dừng.

Explored Set: A, D, G

Path returned: A → D → G

Cost: 2

Bảng tổng hợp so sánh

Thuật toán	Tiêu chí chọn node	Path	Cost
DFS	Độ sâu lớn nhất (LIFO)	A → C → G	3
BFS	Độ sâu nhỏ nhất	A → E	3
UCS	g(n)	A → D → G	2
Greedy	h(n)	A → E	3
A*	g(n)+h(n)	A → D → G	2

Bài 5. Câu hỏi Đúng/Sai

Câu 1: Depth-first search luôn mở rộng ít nhất nhiều node bằng A* sử dụng một admissible heuristic.

→ **Sai**

DFS có thể may mắn đi thẳng tới đích nên mở rộng ít node hơn A*. Ví dụ: S có hai con là G (cost 10) và A (cost 1), A→G cost 1, h=0. Nếu DFS chọn G trước thì nó chỉ mở rộng S rồi G, còn A* phải mở rộng cả S và A trước khi ra G.

Câu 2: Giả sử một quân xe trong cờ vua có thể đi bất kỳ số ô theo hàng hoặc cột trong một nước đi và không thể nhảy qua quân khác. Manhattan distance là một admissible heuristic cho số nước đi nhỏ nhất từ ô A đến ô B.

→ **Sai**

Xe đi được nhiều ô trong một nước nên mọi ô đều tới được trong tối đa 2 nước. Nhưng Manhattan distance có thể rất lớn. Ví dụ từ a1 đến h8, Manhattan là 14 mà chỉ cần 2 nước, nên heuristic đánh giá quá cao, không admissible.

Câu 3: Euclidean distance là một admissible heuristic cho bài toán tìm đường của Pacman.

→ **Đúng**

Pacman đi theo bốn hướng, mỗi bước cost 1. Đường thẳng (Euclid) luôn ≤ độ dài mọi đường đi thực tế, nên không bao giờ đánh giá quá cao.

Câu 4: Tổng của nhiều admissible heuristics vẫn luôn là một admissible heuristic.

→ **Sai**

Tổng có thể vượt chi phí thật. Ví dụ $h_1 = h_2 = h^*$ thì $h_1 + h_2 = 2h^* > h^*$, không admissible.

Câu 5: Admissibility của một heuristic cho A* kéo theo consistency.

→ **Sai**

Admissible không kéo theo consistent. Ví dụ ở 3.3: heuristic admissible nhưng vi phạm điều kiện trên cạnh B→D. (Chiều ngược lại mới đúng: consistent thì admissible.)

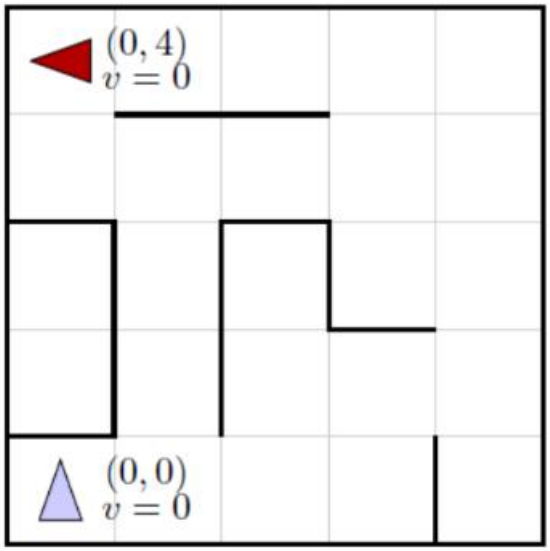
Câu 6: A* với graph search luôn tối ưu nếu heuristic admissible.

→ **Sai**

A* graph search không đảm bảo tối ưu chỉ với heuristic admissible nếu các node đã được đóng không được mở lại. Heuristic consistent là điều kiện đủ để A* graph search đảm bảo tính tối ưu. Phản ví dụ: S→A(1), S→C(3), A→C(1), C→G(3), với h(S)=0, h(A)=4, h(C)=0, h(G)=0. Heuristic này admissible vì h*(A)=4, h*(C)=3. A* đi S→C trước (f=3), đưa C vào explored set, rồi sau đó bỏ qua đường tốt hơn S→A→C. Kết quả là S→C→G cost 6, trong khi tối ưu là S→A→C→G cost 5.

BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Câu 1: Tìm kiếm và heuristic



a) Chuỗi hành động ngắn nhất

Theo quy ước x là cột, y là hàng và (0,0) ở góc dưới trái, đường đi theo nhánh ngoài cùng bên trái là:

(0,0) → (1,0) → (1,3) → (0,3) → (0,4).

Mỗi lần rẽ, xe phải dừng hẳn (v = 0). Chuỗi hành động:

Bước	Hành động	Trạng thái sau hành động
0	Bắt đầu	(0,0,Bắc, 0)
1	Rẽ phải	(0,0,Đông, 0)
2	Tăng tốc	(1,0,Đông, 1)
3	Giảm tốc	(1,0,Đông, 0)
4	Rẽ trái	(1,0,Bắc, 0)
5	Tăng tốc	(1,1,Bắc, 1)
6	Giữ tốc độ	(1,2,Bắc, 1)
7	Giữ tốc độ	(1,3,Bắc, 1)
8	Giảm tốc	(1,3,Bắc, 0)
9	Rẽ trái	(1,3,Tây, 0)
10	Tăng tốc	(0,3,Tây, 1)
11	Giảm tốc	(0,3,Tây, 0)
12	Rẽ phải	(0,3,Bắc, 0)
13	Tăng tốc	(0,4,Bắc, 1)
14	Giảm tốc	(0,4,Bắc, 0)
15	Rẽ trái	(0,4,Tây, 0)

Tổng cộng: 15 hành động. Không thể dùng v = 2 để đi 3 ô trong đoạn này, vì hành động Slower từ v = 2 vẫn làm xe di chuyển thêm 1 ô và sẽ va vào tường tại vị trí kế tiếp.

b) Kích thước không gian trạng thái
Mỗi trạng thái được xác định bởi vị trí (M × N cách), một trong 4 hướng và một trong Vmax + 1 giá trị vận tốc (0, 1, …, Vmax).
Số trạng thái = 4 × M × N × (Vmax + 1).

c) Có thể có trạng thái không có nút con không?
Đáp án: Có.
Ví dụ trạng thái (1, 3, Bắc, v = 2) khi phía trước có tường. Tăng tốc, Giữ tốc độ và Giảm tốc đều khiến xe va vào tường; hai hành động rẽ không hợp lệ vì vận tốc khác 0. Do đó trạng thái này không có nút con hợp lệ.

d) Hệ số phân nhánh lớn nhất
Hệ số phân nhánh lớn nhất là 3.
Ví dụ trạng thái (1, 0, Bắc, v = 0), với các hành động hợp lệ: Tăng tốc, rẽ trái và rẽ phải. Maintain bị cấm khi v = 0; Slower làm vận tốc âm nên không hợp lệ.

e) Manhattan có phải heuristic admissible không?
- Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra là heuristic chấp nhận được, với đúng mê cung 5 × 5 trong đề.
- Manhattan là số ô chênh lệch theo ngang và dọc đến cửa ra.
- Trong mê cung này, mỗi đoạn chạy thẳng dài tối đa 4 ô, và xe phải dừng trước khi rẽ hoặc kết thúc. Với các đoạn ngắn như vậy, số hành động chạy và dừng luôn ít nhất bằng số ô đi được. Rẽ còn tốn thêm hành động.
- Do đó:

$$\text{Manhattan} \leq \text{số hành động tối ưu đến đích}$$

→ **Heuristic admissible.**

f) h = Manhattan / Vmax có admissible không?
Gọi D là khoảng cách Manhattan đến đích.
Mỗi hành động di chuyển tối đa Vmax ô, nên cần ít nhất $\frac{D}{Vmax}$ hành động để đi hết khoảng cách đó.
Việc rẽ và dừng có thể làm tăng số hành động thực tế, do đó $\frac{D}{Vmax}$ không vượt quá chi phí tối ưu.

$$h = \frac{D}{Vmax} \leq h^*$$

→ **Heuristic admissible.**

g) Heuristic không admissible có làm thay đổi tính đầy đủ của A* Tree Search không?
Heuristic không admissible không làm thay đổi tính đầy đủ của A* Tree Search, với heuristic hữu hạn.
Vì đồ thị hữu hạn và h không âm nên h bị chặn. Trên một nhánh vô hạn của cây tìm kiếm, g tăng không giới hạn do mỗi bước tốn ít nhất ε; vì thế f = g + h cũng tăng không giới hạn.
A* sẽ không thể mãi ưu tiên một nhánh vô hạn có chi phí tăng mãi mà bỏ qua mọi nút đích.

h) Heuristic không admissible có làm thay đổi tính tối ưu của A* Tree Search không?
Heuristic không admissible có làm thay đổi tính tối ưu.
Ví dụ có hai đường đến đích:
• S → A → G: chi phí 1 + 1 = 2.
• S → G: chi phí 3.
Nếu heuristic đánh giá quá cao tại A, chẳng hạn h(A) = 10, còn h(G) = 0:

f(A) = 1 + 10 = 11, f(G) = 3 + 0 = 3.

A* chọn đích G trước và trả về đường có chi phí 3, trong khi đường tối ưu chỉ tốn 2.
→ Heuristic không admissible có thể làm A mất tính tối ưu.

i) Lý do hợp lý để dùng heuristic không admissible

☑Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.

☑Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

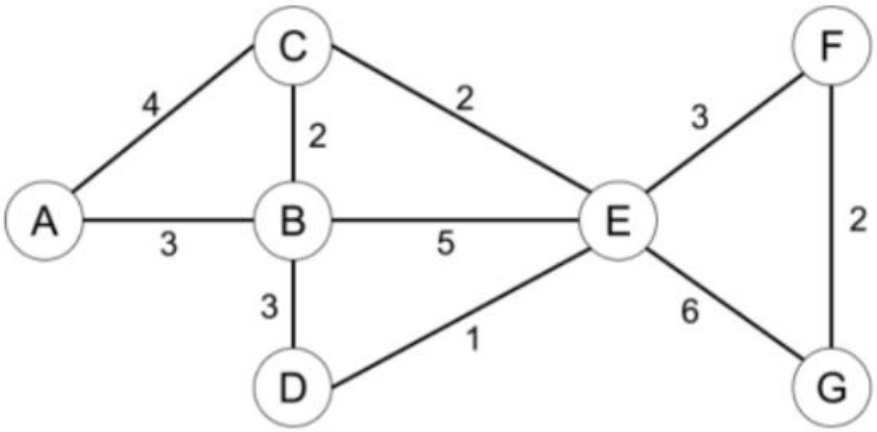
☐ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Câu 2. Các nút tìm kiếm

Xét tree search, không dùng tập explored:
a) BFS: $d(n) < d(n_s)$
BFS mở rộng theo độ sâu tăng dần; các nút được mở rộng trước khi kết thúc nông hơn nút đích nông nhất.
b) UCS: $g(n) < g(n_c)$
UCS mở rộng theo chi phí đường đi tăng dần; trước khi lấy nút đích chi phí thấp nhất n_c, các nút được mở rộng có g nhỏ hơn g(n_c).
c) A* với h consistent: $g(n) + h(n) < g(n_c) + h(n_c)$

A* ưu tiên f(n) = g(n) + h(n). Với heuristic consistent, nút đích chi phí tối ưu được lấy ra khi không còn nút có f nhỏ hơn f(n_c).
Vì h(n_c) = 0, biểu thức cũng có thể viết: $f(n) < g(n_c)$.

Câu 3. Tìm kiếm trên đồ thị



a) Đường đi có thể được trả về

Thuật toán	**A-C-E-G**	**A-C-E-F-G**	**A-B-D-E-F-G**	**Khác**
UCS		X		
Greedy h1				A-B-D-E-G
Greedy h2	X			
A* h1			X	
A* h2		X		

b) Chi phí đường đi tối ưu theo UCS
Đường đi tối ưu: **A → C → E → F → G**.
Chi phí = 4 + 2 + 3 + 2 = 11.

c), d) Kiểm tra admissible và consistent Trước hết, tính chi phí tối ưu thực sự từ mỗi nút đến G:

Nút	Đường tối ưu đến G	h*(n)	$h_1(n)$	$h_2(n)$
A	A-C-E-F-G	11	12	11
B	B-D-E-F-G	9	6	7
C	C-E-F-G	7	9	6
D	D-E-F-G	6	3	4
E	E-F-G	5	3	5
F	F-G	2	2	1
G	G	0	0	0

Vậy: h_1 không admissible.
Chẳng hạn: h_1 (A) = 12 > 11 = h*(A).
Hay h_1 (C) = 9 > 7 = h*(C).
h_1 cũng không Consistent

Xét cạnh A − B, chi phí 3: h_1 (A) = 12 > 3 + h_1 (B) = 9.

Và: h_2 có admissible vì mọi giá trị đều thỏa: h_2 $(n) \leq$ h*(n).

h_2 không consistent, vì:

xét cạnh A − C, chi phí 4: h_2 (A) = 11 > 4 + h_2 (C) = 10.

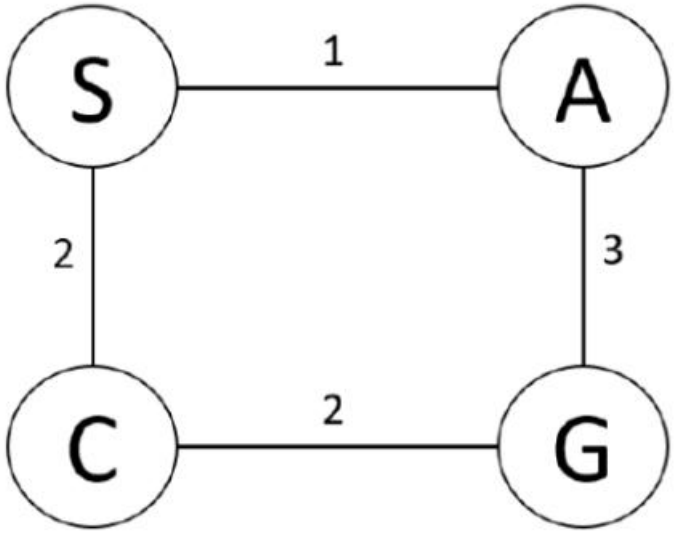
hoặc cạnh E − F: h_2 (E) = 5 > 3 + h_2 (F) = 4.

Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đúng/Sai

Phát biểu	Đáp án	Giải thích ngắn
DFS Tree Search trên đồ thị hữu hạn luôn complete.	**Sai**	Có thể lặp vô hạn theo chu trình trong cây tìm kiếm.
BFS Tree Search trên đồ thị hữu hạn luôn complete.	**Đúng**	BFS duyệt theo độ sâu; nếu có đích ở độ sâu hữu hạn thì sẽ tìm được.
IDS Tree Search trên đồ thị hữu hạn luôn complete.	**Đúng**	Tăng giới hạn độ sâu lần lượt nên sẽ đến độ sâu của một nút đích.
Trên mọi đồ thị không chu trình, Graph Search luôn có frontier lớn hơn Tree Search.	**Sai**	Graph Search loại bỏ các trạng thái lặp; frontier không vì thế mà luôn lớn hơn.
IDS Graph Search có thời gian của BFS và bộ nhớ của DFS.	**Sai**	Graph Search phải lưu tập explored, Tập này có thể chứa rất nhiều trạng thái; vì vậy bộ nhớ không còn như DFS trong trường hợp tổng quát.
Nếu h1 và h2 đều consistent thì min(h1, h2) cũng consistent.	**Đúng**	Điều kiện nhất quán được bảo toàn khi lấy min của hai heuristic consistent.

b) Điền heuristic sao cho heuristic admissible nhưng không consistent



Trạng thái	h(s)	h*(s)
S	4	4
A	2	3

C	2	2
G	0	0

Xét cạnh S → A có chi phí 1:

$$h(S) = 4, \qquad 1 + h(A)=3.$$

Vì:

$$4 > 3 \text{ (Vi phạm điều kiện consistent)}$$

→ Không consistent.

Vậy bộ giá trị trên đáp ứng đúng yêu cầu.